ACADEMIC PROJECT REPORT

Vehicle Service Management System

A MERN-based application for connected customer, vehicle, and service records.

EXECUTIVE SUMMARY

One system for the service story

Vehicle service information is often scattered across paper files, spreadsheets, and disconnected notes. This project proposes a web-based management system that centralizes customers, vehicles, service history, dates, costs, and work details in one controlled interface.

The application uses MongoDB for persistence, Express.js and Node.js for REST APIs, and React for the user interface. Authentication and role-based access help keep operational data visible only to appropriate users.

RECORDS

Customers · Vehicles · Services

STACK

MongoDB · Express · React · Node.js

CONTROL

Authentication · Permissions · Testing

Made with GAMMA

01 · INTRODUCTION

Introduction and project objectives

The system begins with a simple operational problem: service history should be easy to maintain, retrieve, and trust.

PROBLEM STATEMENT

Paper files and disconnected spreadsheets make vehicle records difficult to keep current. Information can be duplicated, misplaced, or separated from the vehicle it describes. Finding a previous repair also depends on manual searching, which slows routine service work and makes consistent record keeping harder.

A shared web application provides a clearer structure. Each service entry can be associated with a specific vehicle and reviewed as part of that vehicle's history.

PROJECT AIM

Demonstrate a complete full-stack workflow

The project connects interface design, API development, database persistence, and access control into one coherent application.

01 *Demonstrate MERN stack development*

02 *Implement MongoDB-backed CRUD operations*

03 *Build REST APIs with Node.js and Express.js*

04 *Develop a React-based user interface*

05 *Support authentication and role-based access where required*

Made with GAMMA

02 · REQUIREMENTS

Requirements and functional scope

The scope is deliberately focused: capture the records that service staff need, then make them practical to retrieve.

CUSTOMERS

People and contact records

Add, view, update, and manage customer records.

VEHICLES

Vehicles linked to owners

Register and maintain vehicle information associated with each customer.

SERVICE RECORDS

Work captured over time

Create and review vehicle service history; track service dates, work performed, and cost.

FUNCTIONAL REQUIREMENTS

- Add, view, update, and manage customer records
- Register and maintain vehicle information
- Create and review vehicle service history
- Track service dates, costs, and work details
- Search or filter records for practical retrieval

NON-FUNCTIONAL PRIORITIES

Usability for routine work, **data consistency** across related records, **security** for controlled access, **maintainability** as the system grows, and **scalability**.

Made with GAMMA

03 · ARCHITECTURE

3. System architecture and data design

A connected request-response system moves from the React client through the API layer to MongoDB persistence.

01

CLIENT

React

Forms, lists, detail views, and edit flows provide the user-facing experience.

02

API LAYER

Express + Node.js

Routes validate requests and coordinate application logic.

03

PERSISTENCE

MongoDB

Documents store customers, vehicles, and service records.

DATA RELATIONSHIPS

Customer

One customer may have multiple vehicles.

Vehicle

Each vehicle may have multiple service records.

Service record

Vehicle reference links each record.

SERVICE RECORD FIELDS

Vehicle reference · service date · work performed · cost · optional notes or status

Made with GAMMA

04 · IMPLEMENTATION

Backend APIs and frontend workflow

The interface turns a user action into a validated request, a database operation, and a visible result.

REST AND CRUD OPERATIONS

REST endpoints expose predictable operations for customers, vehicles, and service records. Create, read, update, and delete actions are separated by resource and handled through the Express and Node.js API layer.

- Customers: create / update to maintain owner records
- Vehicles: read / manage to connect vehicles to customers
- Services: create / review to build vehicle history

REACT INTERFACE

Work follows the record

Forms support data entry; lists and tables make records retrievable. Detail views present a vehicle's service history, while edit flows allow corrections without recreating the full record.

REQUEST FLOW

1. User submits an action in React; React captures a submitted form or selection.
2. Express route validates and processes the request.
3. Node.js service logic reads or writes MongoDB.
4. API returns a response for the interface to display: data or an error.

Made with GAMMA

05 · QUALITY AND ACCESS

5. Authentication, testing, and expected results

Trust depends on both who can access a record and whether the system behaves correctly when it is used.

ACCESS CONTROL

Protected routes

Sign-in establishes an authenticated session before protected operations are available. Role-based permissions can distinguish administrators from service staff, so access follows responsibility rather than convenience.

TESTING APPROACH

- Validate API responses and CRUD behavior
- Test form validation and error states
- Confirm permissions for different user roles
- Verify service history remains linked to the correct vehicle

EXPECTED RESULTS

A usable, connected system

The finished application should centralize service records, reduce manual searching, demonstrate full-stack integration, and present project results clearly. Testing should provide evidence that core operations, validation, permissions, and vehicle-history relationships work as intended.

Made with GAMMA

06 · CONCLUSION

Conclusion and future enhancements

The project closes the loop between a practical service-record problem and a maintainable full-stack solution.

CONCLUSION

The Vehicle Service Management System addresses fragmented records through a connected interface, REST APIs, and MongoDB storage. React gives users a clear path for entering and reviewing information, while Express and Node.js provide the service layer that validates requests and coordinates persistence.

By linking customers to vehicles and vehicles to service records, the design keeps history in context. Authentication, permissions, and testing support the reliability expected from a system used for operational records.

FUTURE SCOPE

01 Automated service reminders

02 Reports and dashboards

03 Billing and invoice management

04 Analytics for service trends and costs

05 Notifications and mobile-friendly workflows

Next-stage principle: extend the system by preserving the same relationships, controlled access, and clear workflow that make the core application dependable.

Made with GAMMA